國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 02

姓名：簡紘田

學號：114310115

Lab01-Ex1. Find maximum
Count operations.
Code

```
#include <iostream>
#include <vector>

using namespace std;
int stepCount = 0;

int findMax(const vector<int>& arr) {

    stepCount = stepCount + 2;      // array access (arr[0]) + assignment (max = arr[0])
    int max = arr[0];

    stepCount++;    // assignment (i = 1)
    for (int i = 1; i < arr.size(); i++) {
        stepCount++;    // comparison (i < arr.size())
        stepCount++;    // array access (arr[i])

        stepCount = stepCount + 2;  // array access (arr[i]) + comparision (arr[i] > max)
        if (arr[i] > max) {

            stepCount = stepCount + 2; // array access (arr[i]) + assignment (max = arr[i])
            max = arr[i];
        }
        stepCount++;    // Increment (i++)
    }
    stepCount++;    // loop exit comparison

    stepCount++;    // return max
    cout << "Step count: " << stepCount << endl;
    return max;
}
```

```
// Total operations:
// 3 (initialization) + 7(n - 1) (loop operations) + 1 (loop exit) + 1 (return)
// = 3 + 7n - 7 + 2
// = 7n - 2 operations
// Therefore, O(n) complexity
```

Lab01-Ex2. Simple Nested Loops

Count operations.

Code

```
#include <iostream>
#include <vector>

using namespace std;
int stepCount = 0;

void printPairs(const vector<int>& arr) {

        stepCount++;    // assignment (i = 0)
        for (int i = 0; i < arr.size(); i++) {
                stepCount++;    // comparison (i < arr.size())


                stepCount += 2; // assignment (j = i + 1) + addition (j = i + 1)
                for (int j = i + 1; j < arr.size(); j++) {
                        stepCount++;    // comparison (j < arr.size())

                        stepCount += 2; // aray access (arr[i])(arr[j])
                        cout << arr[i] << "," << arr[j] << endl;

                        stepCount++;    // increment (j++)
                }
                stepCount++;    //Loop exit comparison
                stepCount++;    //increment (i++)
        }
        stepCount++;    //Loop exit comparison
}
```

```
// Total operations:
// 1 (initialization)
// + n (outer loop comparisons)
// + 2n (inner loop initialization)
// + 2n(n - 1) (inner loop operations)
// + n (inner loop exit comparisons)
// + n (outer loop increments)
// + 1 (outer loop exit)

// = 2n(n - 1) + 5n + 2
// = 2n² - 2n + 5n + 2
// = 2n² + 3n + 2 operations

// Therefore, O(n^2) complexity
```

Lab01-Ex3.

Count operations

Code

Array copying code

```
#include <iostream>
#include <vector>

using namespace std;
int stepCount = 0;


void copyArray(const vector<int>&
source, vector<int>& dest) {

        stepCount++;    // assignment
(i = 0)
        for (int i = 0; i < source.size();
i++) {
                stepCount++;    //
comparison (i < source.size())

                dest[i] = source[i];
```

Array element counter

```
#include <iostream>
#include <vector>

using namespace std;
int stepCount = 0;

int countElement(const vector<int>& arr,
int target) {

        stepCount++;    // assignment
(count = 0)
        int count = 0;

        stepCount++;    // assignment
(i = 0)
        for (int i = 0; i < arr.size();
i++) {
                stepCount++;    //
comparison (i < arr.size())
```

```
                stepCount++;    // increment (i++)
        }
        stepCount++;    // Loop exit comparison
}

// Total operations:
// 1 (initialization) + 5n (loop operations) + 1 (loop exit)

// = 5n + 2 operations

// Therefore, O(n) complexity
```

```
                stepCount += 2; // array access (arr[i]) + comparison (arr[i] == target)
                if (arr[i] == target) {

                        stepCount++;
                        count++;
                }
                stepCount++;    // increment (i++)
        }
        stepCount++;    // loop exit comparison

        stepCount++;    //return count
        return count;
}
// Total operations:
// 2 (initialization) + 5n (loop operations) + 1 (loop exit) + 1 (return)

// = 5n + 4 operations

// Therefore, O(n) complexity
```

Discussion Section

1. Why don't we count operations like arr.size()?

   Because it's typically O(1) in C++ vectors ➔ Constant-timeaccess

   The size is stored as a member variable ➔ No calculations.

2. Why do we drop constants in Big-O notation?

   因為當 n 極大時，常數的影響非常微小

- If an algorithm takes 3n+2 operations, why do we say O(n)?

  因為 2 是常數，在 n 極大時可以捨去，而係數的影響就次方的差距來看，影響也是不大的。

- What's the practical importance of this?

  增加不同演算法處理大量資料時的效率。

3. Looking at the examples above:

- Which has more actual operations: findNumber or sumArray?

  在第一題中，範例的步數為 5n+4 而習題為 7n-2，是 findNumber 多一點

- Does this change their Big O classification? Why or why not?

  不會，因為 n 都仍維持一次，若 n 極大則常數與係數的影響皆小